

Contents

Chapter 1: Geometric Transformation	2
1.1 Homogeneous Coordinates	2
1.2 2D Transformations	2
1.3 3D Rotations	3
Chapter 2: Image Warping and Stitching	3
2.1 Image Warping	3
2.2 Interpolation for Inverse Warping	3
2.3 Panoramas and Image Stitching	4

Chapter 1: Geometric Transformation

Geometric transformations map points from one image (or coordinate frame) to another. They form a strict hierarchy: each class is a special case of the one below it, preserving strictly more geometric structure.

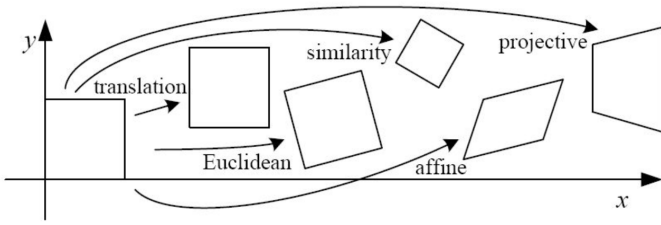
1.1 Homogeneous Coordinates

A 2D point (x, y) is embedded in homogeneous coordinates as the equivalence class $(\tilde{x}, \tilde{y}, \tilde{w}) \sim \lambda(\tilde{x}, \tilde{y}, \tilde{w})$ for any $\lambda \neq 0$, representing the Euclidean point $(x, y) = (\tilde{x}/\tilde{w}, \tilde{y}/\tilde{w})$. The canonical representative uses $\tilde{w} = 1$: $(x, y, 1)^T$. Points at infinity have $\tilde{w} = 0$ and represent directions.

Why homogeneous coordinates? Every transformation in the hierarchy below - including *translation* - becomes a linear (matrix) map on homogeneous coordinates. This allows composing any sequence of transformations by simply multiplying their 3×3 matrices.

1.2 2D Transformations

1.2.1 2D Transformations and Their Hierarchy



Translation (2 DOF.) Shifts the origin; no rotation or distortion.

$$\mathbf{x}' = \mathbf{x} + \mathbf{t}, \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Invariants: all distances, angles, areas, parallelism, and lengths.

Euclidean/Rigid (3 DOF.) Rotation θ plus translation; preserves the shape of objects.

$$\mathbf{x}' = R\mathbf{x} + \mathbf{t}, \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

where R is defined to be the rotation matrix,

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$R \in SO(2)$ satisfies $RR^T = I$, $\det R = 1$. *Invariants:* all distances and angles (congruence).

Similarity (4 DOF.) Adds uniform scaling $s > 0$ to the Euclidean transform.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Invariants: angles, ratios of lengths, shape (but not absolute size). SIFT descriptors are designed to be similarity-invariant.

Affine (6 DOF.) Any invertible linear map plus translation; the last row is fixed to $[0, 0, 1]$.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

where $A \in GL(2)$. An affine map can be decomposed as

$$A = R(\theta) R(-\phi) \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R(\phi)$$

(rotation $\rightarrow$ scale $\rightarrow$ un-rotate). *Invariants:* parallelism, ratios of areas, ratios of lengths along parallel lines. Requires 3 point correspondences to estimate.

Projective / Homography (8 DOF.) The most general linear map on homogeneous coordinates; the last row is free (but H is defined only up to a global scale factor, so 8 free parameters).

$$\tilde{\mathbf{x}}' = H\tilde{\mathbf{x}}, \quad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix},$$

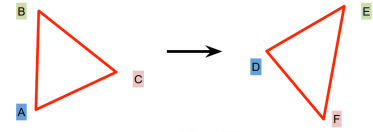
$$\mathbf{x}' = \frac{1}{h_{31}x + h_{32}y + h_{33}} \begin{bmatrix} h_{11}x + h_{12}y + h_{13} \\ h_{21}x + h_{22}y + h_{23} \end{bmatrix}.$$

Invariants: straight lines and cross-ratio of four collinear points. A homography relates any two views of a planar surface (or two views taken from the same camera center). Requires 4 point correspondences to estimate.

Transform	DOF	Matrix	Invariants
Translation	2	$[I \mid \mathbf{t}]$	Distances, angles
Euclidean	3	$[R \mid \mathbf{t}]$	Distances, angles
Similarity	4	$[sR \mid \mathbf{t}]$	Angles, length ratios
Affine	6	$[A \mid \mathbf{t}]$	Parallelism, area ratios
Projective	8	H (full 3×3)	Lines, cross-ratio

1.2.2 How to Determine a 2D Affine Transform

Suppose we have two triangles $\triangle ABC$ and $\triangle A'B'C'$



How do we determine which transformations were applied to $\triangle ABC$ to form $\triangle DEF$? That is, suppose all the points in ABC is $\mathbf{x}$, how do we determine the transformation M such that

$$\mathbf{x}' = M\mathbf{x}$$

Setup. A 2D affine transform in homogeneous coordinates is

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix}}_M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

giving two scalar equations per correspondence:

$$x' = ax + by + t_x, \quad y' = cx + dy + t_y.$$

There are 6 unknowns (a, b, c, d, t_x, t_y) , so **3 point correspondences** (6 equations) are necessary and sufficient — exactly what a triangle provides.

Linear system. Stack the correspondences $A \rightarrow D$, $B \rightarrow E$, $C \rightarrow F$ into the system $P\mathbf{m} = \mathbf{q}$:

$$\underbrace{\begin{bmatrix} x_A & y_A & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_A & y_A & 1 \\ x_B & y_B & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_B & y_B & 1 \\ x_C & y_C & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_C & y_C & 1 \end{bmatrix}}_{\mathbf{m}} \begin{bmatrix} a \\ b \\ c \\ d \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x'_A \\ y'_A \\ x'_B \\ y'_B \\ x'_C \\ y'_C \end{bmatrix}.$$

Solution. Since P is 6×6 and non-singular (invertible) (provided the three source points are not collinear), the unique solution is $\mathbf{m} = P^{-1}\mathbf{q}$. Reconstruct M from $\mathbf{m}$ and the fixed last row.

Note: if the source points are collinear, P is singular and the affine transform is underdetermined — the triangle degenerates and no unique map exists.

More than 3 points: exact vs. noisy. With $n > 3$ correspondences, the system $P\mathbf{m} = \mathbf{q}$ is overdetermined ($P \in \mathbb{R}^{2n \times 6}$). Two cases arise:

- **Noise-free data.** If all n correspondences are exactly consistent with a single affine map (e.g. the corners of a square mapped to a parallelogram), then any 3 non-collinear points already determine M uniquely, and every additional point is automatically satisfied — the extra rows of P are redundant. You can simply pick 3 points and solve the 6×6 system.
- **Noisy data.** Real measurements have noise, so no single M satisfies all equations simultaneously. The system is inconsistent and the best we can do is minimize the sum of squared residuals:

$$\min_{\mathbf{m}} \|P\mathbf{m} - \mathbf{q}\|^2.$$

The solution is given by the **normal equations**:

$$P^T P \mathbf{m} = P^T \mathbf{q} \implies \mathbf{m} = (P^T P)^{-1} P^T \mathbf{q} = P^+ \mathbf{q},$$

where $P^+ = (P^T P)^{-1} P^T$ is the Moore–Penrose pseudoinverse. In practice, solve via QR decomposition or Singular Value Decomposition (SVD) rather than forming $P^T P$ explicitly.

The residual $\|P\mathbf{m}^* - \mathbf{q}\|$ measures how well an affine model fits the data; a large residual suggests the true transform is not affine (e.g. perspective distortion is present).

1.3 3D Rotations

Rotation group $SO(3)$. A 3D rotation is a 3×3 matrix R satisfying $R^T R = I$ and $\det R = +1$. It has 3 DOF. Key representations:

Rotation matrix. Most natural for composing rotations ($R_3 = R_2 R_1$) and rotating vectors ($\mathbf{x}' = R\mathbf{x}$), but 9 numbers for 3 DOF - must maintain orthogonality constraints.

Axis-angle / Rodrigues. Any rotation is a rotation by angle θ about unit axis $\hat{\mathbf{n}}$. Let $[\hat{\mathbf{n}}]_{\times}$ denote the skew-symmetric cross-product matrix. Then:

$$R = I + \sin \theta [\hat{\mathbf{n}}]_{\times} + (1 - \cos \theta) [\hat{\mathbf{n}}]_{\times}^2.$$

Compact (3 parameters: $\theta \hat{\mathbf{n}}$); numerically unstable near $\theta = 0$ or $\theta = \pi$.

Unit quaternion. $q = (q_0, \mathbf{q}) = (\cos(\theta/2), \sin(\theta/2)\hat{\mathbf{n}}) \in \mathbb{R}^4$, $\|q\| = 1$. Rotating a vector: $\mathbf{x}' = q \otimes (0, \mathbf{x}) \otimes q^{-1}$. Composing two rotations: quaternion product $q_2 \otimes q_1$. No gimbal lock; smooth interpolation via SLERP. q and $-q$ represent the same rotation (double cover of $SO(3)$).

3D Rigid Transform (6 DOF). In homogeneous coordinates:

$$\begin{bmatrix} \mathbf{x}' \\ 1 \end{bmatrix} = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad R \in SO(3), \mathbf{t} \in \mathbb{R}^3.$$

This is an element of $SE(3)$, the special Euclidean group. Composition is again matrix multiplication; inversion is $[R^T \mid -R^T \mathbf{t}]$.

Chapter 2: Image Warping and Stitching

2.1 Image Warping

Applying a geometric transformation to an image requires deciding how to compute pixel values at non-integer locations.

Forward warping. For each input pixel (x, y) , compute the output location $\mathbf{x}' = T(\mathbf{x})$ and write the value there.

- Problem: output pixels can be *missed* (holes) or *overwritten* (conflicts) because the map from input to output is not one-to-one on the integer grid.
- Rarely used in practice.

Inverse warping (preferred). For each output pixel $\mathbf{x}'$, compute the source location $\mathbf{x} = T^{-1}(\mathbf{x}')$ in the input and interpolate:

$$g(\mathbf{x}') = f(T^{-1}(\mathbf{x}')).$$

Every output pixel is filled; no holes. Requires T^{-1} to be available (always the case for the five transforms above).

2.2 Interpolation for Inverse Warping

When the inverse map $T^{-1}(\mathbf{x}')$ returns a non-integer location (x, y) , we cannot directly look up a pixel value. Interpolation reconstructs a continuous signal from the discrete grid so that the value at any real-valued coordinate can be estimated.

2.2.1 Nearest-Neighbor Interpolation

The output is assigned the value of the closest grid sample:

$$f(x, y) \approx f(\lfloor x \rfloor, \lfloor y \rfloor),$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer.

- **Cost:** $O(1)$ — one table lookup.
- **Continuity:** C^{-1} (piecewise constant; discontinuities at half-integer boundaries).
- **Artifacts:** Blocky/pixelated appearance, visible staircase edges on diagonal or curved features.
- **Use case:** Label maps, segmentation masks, or any application where the discrete class label must not be blended.

2.2.2 Bilinear Interpolation

Let $(x_0, y_0) = (\lfloor x \rfloor, \lfloor y \rfloor)$ be the integer floor, and define fractional offsets $\alpha = x - x_0$, $\beta = y - y_0 \in [0, 1)$. The four surrounding samples are

$$\begin{aligned} f_{00} &= f(x_0, y_0), & f_{10} &= f(x_0+1, y_0), \\ f_{01} &= f(x_0, y_0+1), & f_{11} &= f(x_0+1, y_0+1). \end{aligned}$$

The bilinear estimate is obtained by first interpolating along rows, then along the column:

$$\begin{aligned} f(x, y_0) &= (1 - \alpha) f_{00} + \alpha f_{10}, \\ f(x, y_0+1) &= (1 - \alpha) f_{01} + \alpha f_{11}, \\ f(x, y) &= (1 - \beta) f(x, y_0) + \beta f(x, y_0+1). \end{aligned}$$

Expanding, the compact form is

$$f(x, y) \approx (1-\alpha)(1-\beta) f_{00} + \alpha(1-\beta) f_{10} + (1-\alpha)\beta f_{01} + \alpha\beta f_{11}.$$

Equivalently, in matrix form:

$$f(x, y) \approx \begin{bmatrix} 1 - \alpha & \alpha \end{bmatrix} \begin{bmatrix} f_{00} & f_{01} \\ f_{10} & f_{11} \end{bmatrix} \begin{bmatrix} 1 - \beta \\ \beta \end{bmatrix}.$$

- **Cost:** 3 lerps (or 4 multiplications + 3 additions per channel).
- **Continuity:** C^0 — the surface is continuous but its partial derivatives are discontinuous at integer boundaries.

- **Artifacts:** Mild blurring; no blocky artifacts. High-frequency detail is attenuated because the interpolating kernel (tent function) has a non-ideal frequency response.
- **Use case:** Default choice for image warping, texture mapping, and image resizing in real-time pipelines.

2.2.3 Bicubic Interpolation

Bicubic interpolation fits a degree-3 polynomial surface through a 4×4 neighborhood of 16 samples. The standard Keys cubic kernel with parameter $a = -0.5$ is

$$w(t) = \begin{cases} (a+2)|t|^3 - (a+3)|t|^2 + 1 & 0 \leq |t| < 1, \\ a|t|^3 - 5a|t|^2 + 8a|t| - 4a & 1 \leq |t| < 2, \\ 0 & |t| \geq 2. \end{cases}$$

The interpolated value is

$$f(x, y) \approx \sum_{m=-1}^2 \sum_{n=-1}^2 f(x_0+m, y_0+n) w(m-\alpha) w(n-\beta).$$

- **Cost:** 16 samples; separable — compute four cubic 1-D interpolations along x , then one along y (12 multiplications per pass).
- **Continuity:** C^1 — both the function and its first derivatives are continuous.
- **Artifacts:** Slight ringing near sharp edges (Gibbs-like overshoot) due to the negative lobes of the kernel.
- **Use case:** High-quality image scaling, photo editing software, and medical imaging.

2.2.4 Sinc / Lanczos Interpolation

The ideal reconstruction filter for a band-limited signal is the sinc:

$$f(x, y) = \sum_m \sum_n f(m, n) \text{sinc}(x-m) \text{sinc}(y-n), \quad \text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}.$$

Because the sinc has infinite support it is truncated and windowed. The Lanczos- a kernel multiplies sinc by a sinc-shaped window of radius a (typically $a = 2$ or 3):

$$L_a(t) = \begin{cases} \text{sinc}(t) \text{sinc}(t/a) & |t| < a, \\ 0 & |t| \geq a. \end{cases}$$

- **Cost:** $(2a)^2$ samples; $a = 3$ requires 36 samples per output pixel.
- **Continuity:** C^0 at window boundary; approximately C^∞ elsewhere.
- **Artifacts:** Best preservation of fine detail and frequency content; some ringing possible for $a = 2$.
- **Use case:** Offline high-quality downsampling and super-resolution pre-processing.

2.2.5 Comparison and Practical Guidelines

Method	Support	Continuity	Sharpness	Speed
Nearest-neighbor	1×1	C^{-1}	Highest (aliased)	Fastest
Bilinear	2×2	C^0	Medium	Fast
Bicubic	4×4	C^1	High	Moderate
Lanczos-3	6×6	$\approx C^\infty$	Highest (clean)	Slow

Key design decisions:

- **Upsampling** (output larger than input): bicubic or Lanczos avoids the blurring of bilinear.
- **Downsampling** (output smaller than input): always *anti-alias first* by low-pass filtering before sampling; otherwise all methods alias.
- **Discrete labels** (segmentation, depth maps): nearest-neighbor is mandatory to prevent blending across class boundaries.
- **Real-time constraints:** bilinear is hardware-accelerated in every GPU texture unit and is the pragmatic default.

2.3 Panoramas and Image Stitching



Figure 1: Alcatraz Island, shown in a panorama created by image stitching

Overview. Panorama creation stitches overlapping photographs captured from (approximately) the same optical centre into a single wide-angle composite. Standard pipeline: **(1)** detect and match features across image pairs; **(2)** robustly estimate a homography per pair via RANSAC+DLT; **(3)** project all images onto a common surface (plane, cylinder, or sphere); **(4)** warp and composite; **(5)** blend seams. A homography relates two images whenever the camera undergoes a *pure rotation* (fixed optical centre) or the scene is *planar*.

Homography Estimation - Direct Linear Transform (DLT). Given $n \geq 4$ point correspondences $\{\mathbf{x}_i \leftrightarrow \mathbf{x}'_i\}$, DLT finds H by forming a homogeneous linear system. Since $\tilde{\mathbf{x}}'_i \sim H\tilde{\mathbf{x}}_i$ (homogeneous equality), the cross-product condition

$$\tilde{\mathbf{x}}'_i \times H\tilde{\mathbf{x}}_i = \mathbf{0}$$

expands (for $\tilde{\mathbf{x}}_i = (x_i, y_i, 1)^T$ and $\tilde{\mathbf{x}}'_i = (x'_i, y'_i, 1)^T$) to two independent equations:

$$A_i \mathbf{h} = \mathbf{0}, \quad A_i = \begin{bmatrix} \mathbf{0}^T & -\tilde{\mathbf{x}}_i^T & y'_i \tilde{\mathbf{x}}_i^T \\ \tilde{\mathbf{x}}_i^T & \mathbf{0}^T & -x'_i \tilde{\mathbf{x}}_i^T \end{bmatrix} \in \mathbb{R}^{2 \times 9},$$

where $\mathbf{h} = (h_{11}, h_{12}, \dots, h_{33})^T \in \mathbb{R}^9$. Stacking all n correspondences:

$$A\mathbf{h} = \mathbf{0}, \quad A \in \mathbb{R}^{2n \times 9}.$$

The solution is the *right singular vector* of A for the *smallest singular value* (via SVD), minimizing $\|A\mathbf{h}\|$ subject to $\|\mathbf{h}\| = 1$. With $n = 4$ correspondences A is 8×9 , the null space is one-dimensional, and H is determined up to scale (8 DOF).

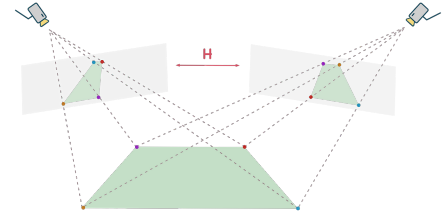


Figure 2: Homography Estimation

Normalization (essential). Raw pixel coordinates (spanning hundreds to thousands of units) make A severely ill-conditioned. Before forming A , independently normalize each point set:

1. Translate so the centroid is at the origin:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \boldsymbol{\mu}$$

2. Scale uniformly so the mean distance from the origin is $\sqrt{2}$:

$$s = \sqrt{2 / \|\tilde{\mathbf{x}}_i\|}$$

3. The normalisation matrix is

$$T = \begin{bmatrix} s & 0 & -s\mu_x \\ 0 & s & -s\mu_y \\ 0 & 0 & 1 \end{bmatrix}$$

Run DLT on $\hat{\mathbf{x}}_i = T\mathbf{x}_i$ and $\hat{\mathbf{x}}'_i = T'\mathbf{x}'_i$ to obtain $\hat{H}$; then denormalise: $\bar{H} = T'^{-1}\hat{H}T$. Normalisation reduces condition numbers by orders of magnitude and is mandatory for numerical reliability (Hartley, 1997).

RANSAC for robust estimation. Descriptor matching yields many outliers. RANSAC applies with minimum sample $s = 4$ (to fit H) and inlier threshold $\epsilon \approx 3$ pixels on reprojection error $\|\mathbf{x}'_i - H\mathbf{x}_i\|$. After RANSAC, refit H on all inliers via DLT. Required iterations: $k = \log(1-p) / \log(1-(1-e)^4)$; for $e = 0.5$, $p = 0.99$: $k \approx 72$.

Non-linear refinement. DLT minimises an *algebraic* error with no geometric meaning. For higher accuracy, refine H by minimising the **symmetric reprojection error**:

$$\min_H \sum_i \left(d(\mathbf{x}'_i, H\mathbf{x}_i)^2 + d(\mathbf{x}_i, H^{-1}\mathbf{x}'_i)^2 \right),$$

solved by Levenberg–Marquardt initialised from the DLT solution.

Cylindrical projection. For wide-angle panoramas (> 60 field of view), projective stitching produces severe distortion near the periphery. Instead, project each image onto a *cylinder* of radius f (focal length in pixels). For a point (x, y) with origin at the image center:

$$\hat{x} = f \arctan\left(\frac{x}{f}\right), \quad \hat{y} = \frac{fy}{\sqrt{x^2 + f^2}}.$$

In cylindrical coordinates, a pure camera rotation about the vertical axis becomes a *horizontal translation*, making alignment trivial for rotation-only sequences. The focal length f must be known (from calibration or EXIF).

Spherical projection. For full 360×180 panoramas, project onto a sphere of radius f . Spherical angles of image point (x, y) :

$$\phi = \arctan\left(\frac{x}{f}\right), \quad \lambda = \arctan\left(\frac{y}{\sqrt{x^2 + f^2}}\right).$$

Re-project to *quirectangular* (longitude–latitude) format: $(\phi, \lambda) \rightarrow (f\phi, f\lambda)$. Used in 360° video, VR headsets, and Google Street View.

Image blending. Warped images must be composited; seams appear due to exposure differences, vignetting, and sub-pixel misalignment.

Alpha feathering. Assign each pixel a weight $w(\mathbf{x})$ proportional to its distance from the image boundary. In overlap regions blend linearly:

$$C(\mathbf{x}) = \frac{w_A(\mathbf{x}) C_A(\mathbf{x}) + w_B(\mathbf{x}) C_B(\mathbf{x})}{w_A(\mathbf{x}) + w_B(\mathbf{x})}.$$

Simple to implement; can produce ghosting under misalignment.

Multiband blending (Burt & Adelson, 1983). Low-frequency content should be blended with a wide transition (to hide color shifts); high-frequency content should be cut sharply (to preserve edge sharpness). Algorithm:

1. Build Gaussian pyramids $G_A^{(\ell)}$, $G_B^{(\ell)}$ and mask pyramid $G_M^{(\ell)}$.
2. Build Laplacian pyramids: $L_A^{(\ell)} = G_A^{(\ell)} - \text{up}(G_A^{(\ell+1)})$, similarly $L_B^{(\ell)}$.
3. Blend each level: $L_S^{(\ell)} = G_M^{(\ell)} \cdot L_A^{(\ell)} + (1 - G_M^{(\ell)}) \cdot L_B^{(\ell)}$.
4. Collapse the blended Laplacian pyramid to reconstruct the composite.

Seam finding (graph cut). Rather than blending the entire overlap, find a *minimum-cost cut* through the overlap where the two images agree most closely (minimum photometric difference). Kwatra et al. (2003) formulate this as a Markov Random Field and solve with graph cuts, eliminating ghosting entirely.

Bundle adjustment for multi-image panoramas. With $m > 2$ images, composing pairwise homographies accumulates drift. Global alignment parameterises each camera by its rotation $R_i \in SO(3)$ and focal length f_i , then jointly minimises total reprojection error over all matched image pairs (i, j) and inlier correspondences k :

$$\min_{\{R_i, f_i\}} \sum_{(i,j) \in \mathcal{E}} \sum_{k \in \mathcal{M}_{ij}} \left\| \mathbf{x}_k^{(j)} - \pi\left(R_j R_i^{-1} \pi^{-1}\left(\mathbf{x}_k^{(i)}; f_i\right); f_j\right) \right\|^2,$$

where $\pi(\cdot; f)$ is projection with focal length f and π^{-1} back-projects to 3D. Solved by Levenberg–Marquardt (AutoStitch, Brown & Lowe, 2007). **Gain compensation** (per-image exposure scale g_i) is typically added to the objective to handle photometric inconsistency between images.